最短距离 Dijkstra 算法和最小生成树 prim 算法对比

陈斌 北京大学

最短距离 Dijkstra 算法和最小生成树 prim 算法的区别非常相似，稍不留意就会造成混淆。

首先，两个算法都是利用优先队列实现，都是典型的贪心策略算法。

其次，都是以一个无向图 G，起始定点 start，作为参数。

其算法程序框架几乎一样，不同点如下：

1. Dijkstra 算法利用节点的 dist 属性来记录节点到起始节点的最短权重距离，而 prim 算法则利用节点的 dist 属性来记录节点到已建树节点集合的最小权重代价；

2. Dijkstra 算法每次从优先队列提取的是到起始节点最短权重距离的节点作为当前节点，而 prim 算法每次从优先队列提取的是到已建树节点集合最小权重代价的节点作为当前节点；

3. Dijkstra 算法遍历当前节点的每一个邻接节点，如果当前节点最短权重距离加上邻接边权重，比邻接节点已有的最短权重距离还短，那就更新邻接节点的最短权重距离和前驱；

prim 算法遍历当前节点的每一个安全邻接节点（不在已建树节点集合里），如果当前节点到安全邻接节点的权重，比安全邻接节点已有的最小权重代价还小，那就更新安全邻接节点的最小权重代价和前驱；

4. Dijkstra 算法执行完毕后，每个节点的 dist 记录了到起始节点的最短距离，pred 记录了最短距离路径的前驱节点；

prim 算法执行完毕后，每个节点的 dist 记录了到前驱节点的边权重，pred 记录了最小生成树路径的前驱节点，把所有 dist 求和，就是最小生成树的权重代价。

1 Dijkstra 算法加详细注释的代码和测试结果

```
import sys


class Graph:
    def __init__(self):
        self.vertices = {}
        self.numVertices = 0

    def addVertex(self, key):
        self.numVertices = self.numVertices + 1
        newVertex = Vertex(key)
        self.vertices[key] = newVertex
        return newVertex

    def getVertex(self, n):
        if n in self.vertices:
            return self.vertices[n]
        else:
            return None

    def __contains__(self, n):
        return n in self.vertices

    def addEdge(self, f, t, cost=0):
        if f not in self.vertices:
            nv = self.addVertex(f)
        if t not in self.vertices:
            nv = self.addVertex(t)
        self.vertices[f].addNeighbor(self.vertices[t], cost)

    def getVertices(self):
        return list(self.vertices.keys())

    def __iter__(self):
        return iter(self.vertices.values())

    def __str__(self):
        s = ""
        for v in self:
            s += f"[{v.id},{v.dist},{v.pred.id if v.pred else None}] "
        return s


```

```
class Vertex:
    def __init__(self, num):
        self.id = num
        self.connectedTo = {}
        self.color = 'white'
        self.dist = sys.maxsize
        self.pred = None
        self.disc = 0
        self.fin = 0

    # def __lt__(self,o):
    #     return self.id < o.id

    def addNeighbor(self, nbr, weight=0):
        self.connectedTo[nbr] = weight

    def setColor(self, color):
        self.color = color

    def setDistance(self, d):
        self.dist = d

    def setPred(self, p):
        self.pred = p

    def setDiscovery(self, dtime):
        self.disc = dtime

    def setFinish(self, ftime):
        self.fin = ftime

    def getFinish(self):
        return self.fin

    def getDiscovery(self):
        return self.disc

    def getPred(self):
        return self.pred

    def getDistance(self):
        return self.dist

    def getColor(self):
        return self.color

```

```
90      def getConnections(self):
91          return self.connectedTo.keys()
92
93      def getWeight(self, nbr):
94          return self.connectedTo[nbr]
95
96      def __str__(self):
97          return str(self.id) + ":color " + self.color + ":disc " + str(self.disc) ↩
                + ":fin " + str(
98              self.fin) + ":dist " + str(self.dist) + ":pred \n\t[" + str(self.pred↩
                    ) + "]\n"
99
100     def __repr__(self):
101         return str(self.id)
102
103     def getId(self):
104         return self.id
105
106
107 class PriorityQueue:
108     def __init__(self):
109         self.heapArray = [(0, 0)]
110         self.currentSize = 0
111
112     def buildHeap(self, alist):
113         self.currentSize = len(alist)
114         self.heapArray = [(0, 0)]
115         for i in alist:
116             self.heapArray.append(i)
117         i = len(alist) // 2
118         while (i > 0):
119             self.percDown(i)
120             i = i - 1
121
122     def percDown(self, i):
123         while (i * 2) <= self.currentSize:
124             mc = self.minChild(i)
125             if self.heapArray[i][0] > self.heapArray[mc][0]:
126                 tmp = self.heapArray[i]
127                 self.heapArray[i] = self.heapArray[mc]
128                 self.heapArray[mc] = tmp
129             i = mc
130
131     def minChild(self, i):
132         if i * 2 > self.currentSize:
133             return -1
```

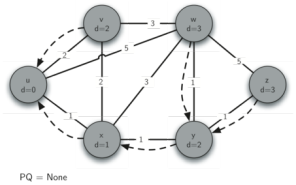
```
134         else:
135             if i * 2 + 1 > self.currentSize:
136                 return i * 2
137             else:
138                 if self.heapArray[i * 2][0] < self.heapArray[i * 2 + 1][0]:
139                     return i * 2
140                 else:
141                     return i * 2 + 1
142 
143     def percUp(self, i):
144         while i // 2 > 0:
145             if self.heapArray[i][0] < self.heapArray[i // 2][0]:
146                 tmp = self.heapArray[i // 2]
147                 self.heapArray[i // 2] = self.heapArray[i]
148                 self.heapArray[i] = tmp
149             i = i // 2
150 
151     def add(self, k):
152         self.heapArray.append(k)
153         self.currentSize = self.currentSize + 1
154         self.percUp(self.currentSize)
155 
156     def delMin(self):
157         retval = self.heapArray[1][1]
158         self.heapArray[1] = self.heapArray[self.currentSize]
159         self.currentSize = self.currentSize - 1
160         self.heapArray.pop()
161         self.percDown(1)
162         return retval
163 
164     def isEmpty(self):
165         if self.currentSize == 0:
166             return True
167         else:
168             return False
169 
170     def decreaseKey(self, val, amt):
171         # this is a little wierd, but we need to find the heap thing to decrease ↩
            by
172         # looking at its value
173         done = False
174         i = 1
175         myKey = 0
176         while not done and i <= self.currentSize:
177             if self.heapArray[i][1] == val:
178                 done = True
```

```
                myKey = i
            else:
                i = i + 1
        if myKey > 0:
            self.heapArray[myKey] = (amt, self.heapArray[myKey][1])
            self.percUp(myKey)

    def __contains__(self, vtx):
        for pair in self.heapArray:
            if pair[1] == vtx:
                return True
        return False

    def __str__(self):
        return str(self.heapArray)
```

```
from pq import PriorityQueue, Graph, Vertex


# 最短路径
# G - 无向赋权图
# start - 开始节点
# 返回从开始节点到其它所有节点的最短带权路径
def dijkstra(G, start):
    pq = PriorityQueue()  # 创建优先队列
    start.setDistance(0)  # 起点距离设置为0，其它节点距离默认maxsize
    # 将节点排入优先队列，start在最前面
    pq.buildHeap([(v.getDistance(), v) for v in G])
    while not pq.isEmpty():
        # 取从start开始距离最小的节点出队列，作为当前节点
        # 当前节点已解出最短路径
        currentVert = pq.delMin()
        # 遍历节点的所有邻接节点
        for nextVert in currentVert.getConnections():
            # 从当前节点出发，逐个加上邻接节点的距离进行检验
            newDist = currentVert.getDistance() \
                      + currentVert.getWeight(nextVert)
            # 如果小于邻接节点原有距离，就更新邻接节点
            if newDist < nextVert.getDistance():
                # 更新距离值
                nextVert.setDistance(newDist)
                # 更新返回路径
                nextVert.setPred(currentVert)
                # 更新优先队列
```

```
                pq.decreaseKey(nextVert, newDist)


G = Graph()
ndedge = [('u', 'v', 2), ('u', 'w', 5), ('u', 'x', 1),
          ('v', 'x', 2), ('v', 'w', 3), ('x', 'w', 3),
          ('x', 'y', 1), ('w', 'y', 1), ('w', 'z', 5),
          ('y', 'z', 1)]
for nd in ndedge:
    G.addEdge(nd[0], nd[1], nd[2])
    G.addEdge(nd[1], nd[0], nd[2])
start = G.getVertex('u')
dijkstra(G, start)
print(G)
```

测试用图是：



执行结果是：

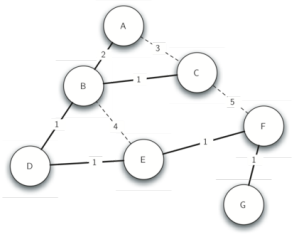
```
        [u,0,None] [v,2,u] [w,3,y] [x,1,u] [y,2,x] [z,3,y]
```

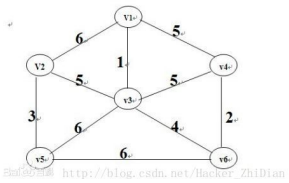
2 prim 算法加详细注释的代码和测试结果

```
from pq import PriorityQueue, Graph, Vertex


# 最小生成树prim算法
# G - 无向赋权图
# start - 开始节点
# 返回从开始节点创建最小生成树
def prim(G, start):
    pq = PriorityQueue()  # 创建优先队列
    start.setDistance(0)  # 起点最小权重代价设置为0，其它节点最小权重代价默认maxsize
    # 将节点排入优先队列，start在最前面
    pq.buildHeap([(v.getDistance(), v) for v in G])
    while not pq.isEmpty():
        # 取距离*已有树*最小权重代价的节点出队列，作为当前节点
        # 当前节点已解出最小生成树的前驱pred和对应最小权重代价dist
        currentVert = pq.delMin()
        # 遍历节点的所有邻接节点
        for nextVert in currentVert.getConnections():
            # 从当前节点出发，逐个检验到邻接节点的权重
            newCost = currentVert.getWeight(nextVert)
            # 如果邻接节点是"安全边"，并且小于邻接节点原有最小权重代价dist，就更新邻接节点
            if nextVert in pq and newCost < nextVert.getDistance():
                # 更新最小权重代价dist
                nextVert.setPred(currentVert)
                # 更新返回路径
                nextVert.setDistance(newCost)
                # 更新优先队列
                pq.decreaseKey(nextVert, newCost)


G = Graph()
ndedge = [('A', 'B', 2), ('A', 'C', 3), ('B', 'C', 1),
          ('B', 'D', 1), ('B', 'E', 4), ('C', 'F', 5),
          ('D', 'E', 1), ('E', 'F', 1), ('F', 'G', 1)]
for nd in ndedge:
    G.addEdge(nd[0], nd[1], nd[2])
    G.addEdge(nd[1], nd[0], nd[2])
start = G.getVertex('A')
prim(G, start)
print(G)

```

```
G = Graph()
ndedge = [('v1', 'v2', 6), ('v1', 'v3', 1), ('v1', 'v4', 5),
          ('v2', 'v3', 5), ('v3', 'v4', 5), ('v2', 'v5', 3),
          ('v3', 'v5', 6), ('v3', 'v6', 4), ('v4', 'v6', 2),
          ('v5', 'v6', 6)]
for nd in ndedge:
    G.addEdge(nd[0], nd[1], nd[2])
    G.addEdge(nd[1], nd[0], nd[2])
start = G.getVertex('v1')
prim(G, start)
print(G)
```

测试用图是：





执行结果是：

```
        [A,0,None] [B,2,A] [C,1,B] [D,1,B] [E,1,D] [F,1,E] [G,1,F]
        [v1,0,None] [v2,5,v3] [v3,1,v1] [v4,2,v6] [v5,3,v2] [v6,4,v3]
```